Zadanie 2: optymalizacja z ograniczeniami

Termin realizacji: 30 marca 2026

Wybierz funkcję testową wykorzystaną w zadaniu 1.

Zadanie do oddania przez MS Teams. Do oddania: kod oraz krótkie sprawozdanie w PDF (można na przykład przy użyciu `quarto render notebook.ipynb --to pdf`).

Na 3.0

Do realizacji:

1. Dodaj ograniczenie postaci $x_1^2 + x_2 + b = 0$ ze stałą b dopasowaną w taki sposób, aby żadne minimum lokalne (przynajmniej w zakresie w którym losowany jest punkt początkowy) nie spełniało ograniczenia.
2. Zaimplementuj metodę funkcji kary do rozwiązania tego problemu. Zwróć uwagę, aby metoda wykonywała częściową optymalizację dla każdej wielkości parametru kary μ_k (wykonanie od 1 do 5 kroków dla ustalonej wartości parametru kary – do samodzielnego wyboru).
3. Wylosuj 10 punktów z dziedziny przeszukiwania z tabelki. Dla każdego z nich przeprowadź 100 kroków optymalizacji metodą największego spadku ze stałym krokiem. Narysuj wykres zależności wartości funkcji optymalizowanej od kroku optymalizacji.
4. Wykorzystaj ponownie metodę optymalizacji z poprzedniego projektu. Jako funkcję celu i jej gradient przekaż funkcje uwzględniające karę (ogólny schemat):

```
function penalty_gradient_descent(f, f_grad, x0; ...)
    x_k = copy(x0)
    for i in 1:max_iter
        # dodać: wyliczanie współczynnika kary
        function penalized_f(x)
            # do uzupełnienia
        end
        function penalized_f_grad(x)
```

```
            # do uzupełnienia
        end
        x_k = gradient_descent(penalized_f, penalized_f_grad, x_k; max_iter=3)
    end
end
```

5. Przeprowadź procedurę dla kilkunastu kroków. Spróbuj zilustrować brak zbieżności, szybką zbieżność i powolną zbieżność.

Na 4.0

Do realizacji:

1. Punkty z zadania na 3.0.
2. Zamień metodę największego spadku na metodę gradientów sprzężonych. Metoda ta powinna wykorzystywać przeszukiwanie w kierunku uwzględniające regułę Armijo.
3. Wraz ze zwiększaniem się parametru kary powinna maleć norma gradientu przy której kończymy działanie metody gradientów sprzężonych, na przykład może być równa $10^{-2}/\mu_k$.

Na 5.0

Do realizacji:

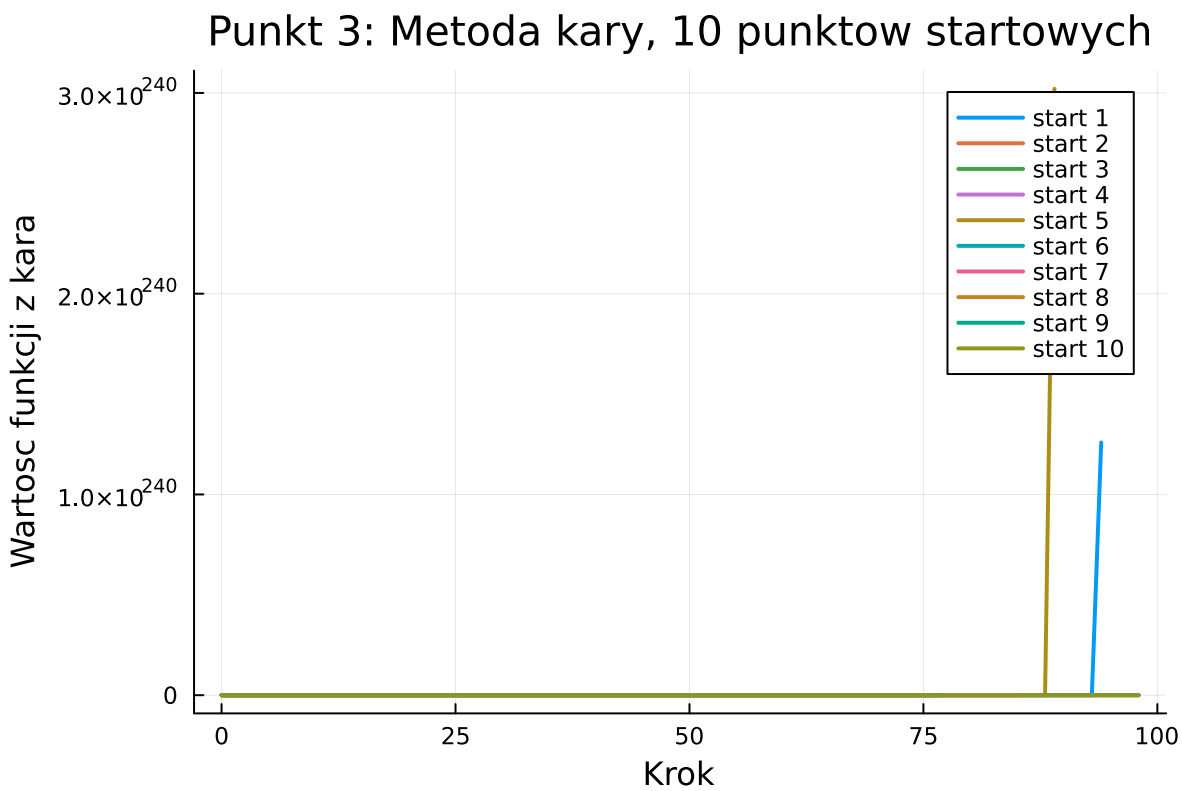
1. Punkty z zadania na 4.0.
2. Wykonaj benchmarking metody z użyciem `BenchmarkTools.jl`. Zanotuj czasy działania wywołań optymalizacji oraz liczbę alokacji. Spróbuj zoptymalizować działanie funkcji korzystając wymienionych tu rad: Julia performance tips. W sprawozdaniu napisz jakie zmiany wykonane i jak wpłynęły na czas działania programu.

Uwagi:

1. Dodanie typów w nagłówku metody przyspiesza kod tylko w rzadkich i bardzo szczególnych sytuacjach, i to tylko dla typów reprezentujących typy (`Core.Type`) lub funkcje.
2. Dobrym pomysłem jest wykonywanie operacji na tablicach w miejscu i unikanie alokacji. Warto też zwrócić uwagę na tzw. *type stability*, czyli unikanie sytuacji w których typ zmiennej zależy od wartości innej zmiennej.

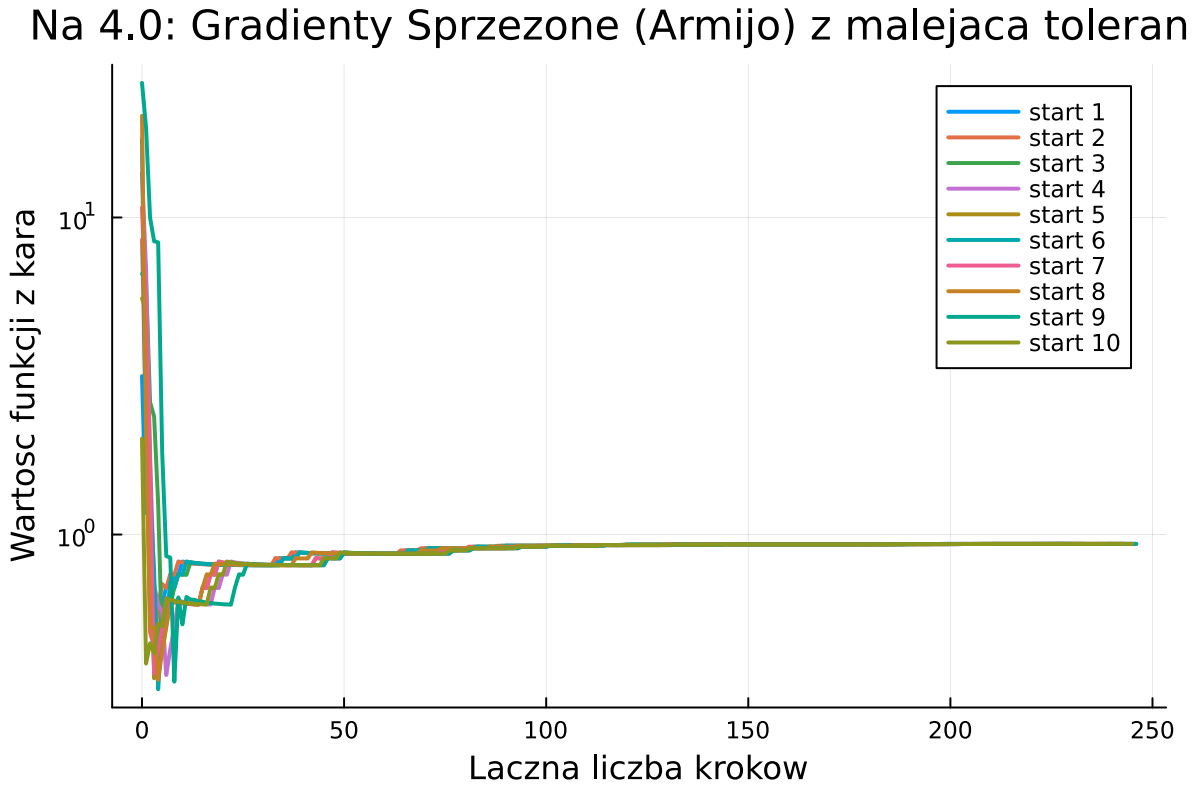
Na 3.0

```
Sprawdzenie ograniczenia w minimum Three Hump Camel: g([0,0]) = 1.0
```




```
Punkt 4: przykład 3 kroków gradient_descent na funkcji z karą
x0 = [2.5, -3.0]
x_po_3_krokach = [1.9921469694578735, -3.0846293336646644]
historia P(x) = [103.98697916666666, 52.7124490455571, 32.838030714732184, 22.93499850193334
```

Na 4.0



Na 5.0

Najpierw przeprowadziliśmy benchmarking obecnego rozwiązania z gradientami sprzeżonymi

```
BenchmarkTools.Trial: 1631 samples with 1 evaluation per sample.
 Range (min … max):  2.286 ms … 62.646 ms    GC (min … max):  0.00% … 94.59%
 Time  (median):     2.368 ms               GC (median):     0.00%
 Time  (mean ± ):   3.061 ms ±  3.042 ms    GC (mean ± ):  14.89% ± 14.33%


  2.29 ms      Histogram: log(frequency) by time     15.4 ms <

 Memory estimate: 3.73 MiB, allocs estimate: 90644.
```

Potem przeprowadziliśmy optymalizacje dla armijo search i udało nam się zredukować liczbę alokacji do zera

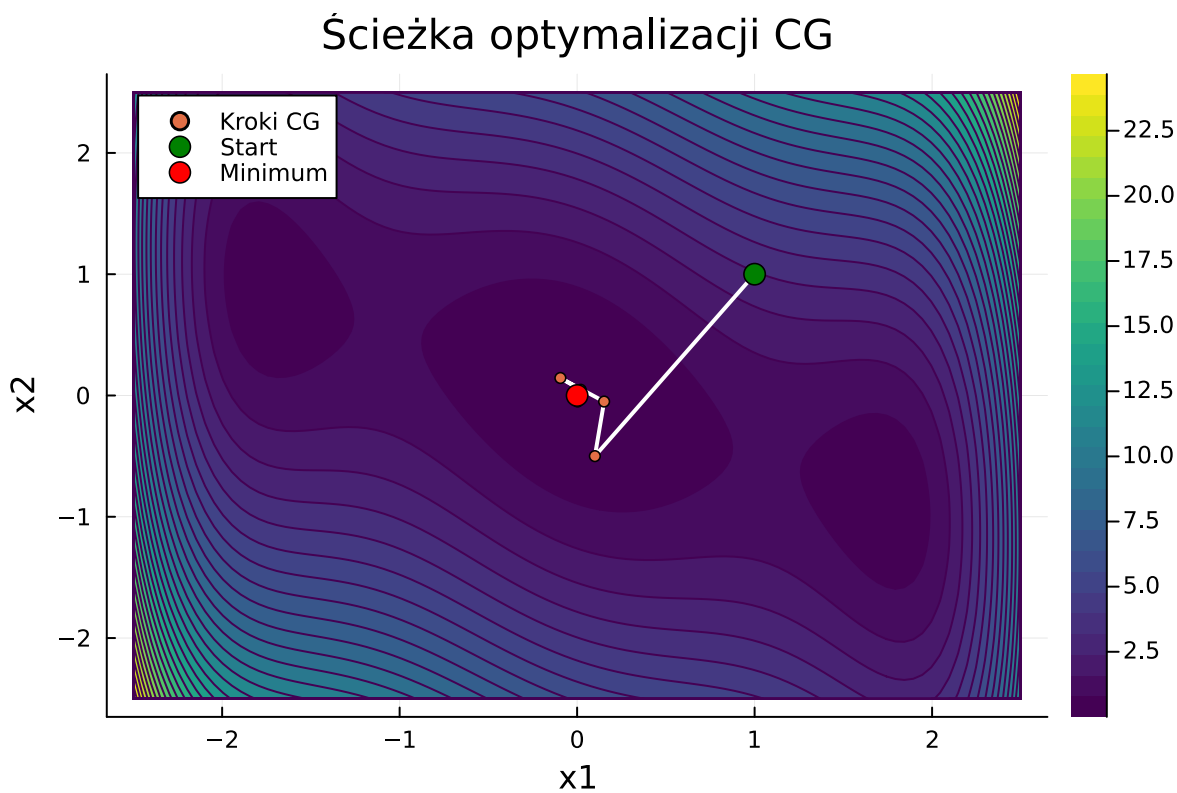
```
86.931 ns (4 allocations: 160 bytes)
57.456 ns (0 allocations: 0 bytes)
```

Potem zoptymalizowaliśmy funkcję gradientową, żeby nie alokowała pamięci

```
342.280 ns (5 allocations: 240 bytes)
35.866 ns (0 allocations: 0 bytes)
```

Potem zoptymalizowaliśmy gradienty sprzężone, pozostawiając tylko jedną alokację

```
Wynik optymalizacji: [-5.336291901165819e-7, 6.554093904458986e-7]
Liczba iteracji: 15
Wartość funkcji w minimum: 6.493361119550899e-13
```

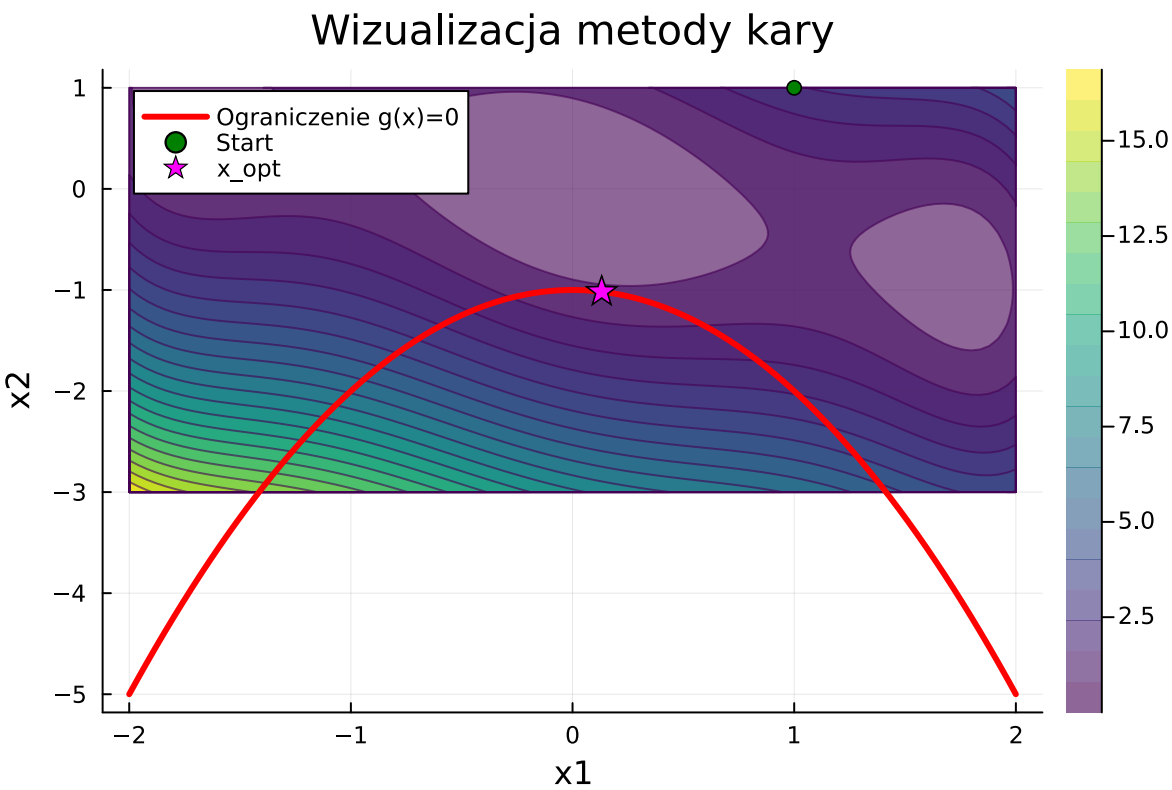


```
Benchmark (bez historii):
  65.995 ns (1 allocation: 16 bytes)
```

Kolejnym krokiem była optymalizacja całej pętli optymalizacji z ograniczeniami

```
  142.743  s (39 allocations: 944 bytes)

--- TEST JAKOŚCIOWY ---
Punkt startowy: [1.0, 1.0]
Punkt końcowy (x_opt): [0.13154054501852647, -1.0172992851880052]
Wartość funkcji celu f(x_opt): 0.9353740661698178
Wartość ograniczenia g(x_opt): 3.629795765847099e-6
Ostatnie mu: 524288.0
Sukces: Ograniczenie jest spełnione (bliskie 0).
```



Widać trzykrotną redukcję czasu i znaczną redukcję pamięci

Średni czas wychodzi ~800 mikrosekund zamiast 2,9 ms

```
BenchmarkTools.Trial: 5851 samples with 1 evaluation per sample.
 Range (min … max):  792.542 us …   2.189 ms    GC (min … max): 0.00% … 0.00%
 Time  (median):     805.869 us                GC (median):    0.00%
 Time  (mean ± ):   852.657 us ± 172.106 us    GC (mean ± ):  0.00% ± 0.00%



  793 us        Histogram: log(frequency) by time       1.85 ms <

 Memory estimate: 8.44 KiB, allocs estimate: 340.
```

Wnioski z zadania 5:

- Robienie obliczeń w miejscu i usuwanie niepotrzebnych alokacji jest skuteczne w zmniejszaniu czasu wykonania programu